

WASI P3 Vote

June 10, 2026

Agenda

- Phase criteria and Consensus reminders
- Wasmtime implementation status
- JCO implementation status
- Vote: Include **wasi-cli** in WASIP3
- Vote: Include **wasi-http** in WASIP3
- Vote: Launch **WASI 0.3.0**

Process

We held the vote to advance WASI 0.2.0 **Jan 25, 2024**.

For 0.3.0, we will follow the **exact same process** established for 0.2.0.

Additive, non-breaking changes may be made to proposals in patch releases.

We will begin the 0.3.x patch release train once we vote to launch 0.3.0.

Where we are

- WASI P2 (0.2.12) released **June 9, 2026**
- WASI P3 (0.3.0) drafts and release candidates
 - Drafts began **Mar 8, 2024**
 - 0.3.0-rc-2025-08-15
 - 0.3.0-rc-2025-09-16
 - 0.3.0-rc-2025-11-10
 - 0.3.0-rc-2026-01-06
 - 0.3.0-rc-2026-02-05
 - 0.3.0-rc-2026-02-09
 - **0.3.0-rc-2026-03-15**
- WASI P3's included proposals are **breaking revisions** of (phase 3) proposals already shipped in WASI P2

Vote to Include in WASIP3

- To be included in WASIP3, a proposal must:
 - a. Reach phase 3 in the WASI Subgroup Phase Process
 - b. Satisfy its own portability criteria
 - c. Be voted for inclusion by the WASI Subgroup
- Proposals may be added to WASIP3 at any time
(after meeting the above criteria)
until WASI reaches 1.0

Phase 3 Criteria

2. Feature Description Available [WASI Subgroup]

Entry requirements:

- The portability criteria are documented in the proposal.
- Precise and complete overview document is available in a proposal repo around which a reasonably high level of consensus exists.
- A [wit](#) description of the API exists.
- All dependencies of the wit description must have reached phase 2.

During this phase:

- One or more implementations proceed on prototyping the API.
- A plan is developed for how the portability criteria will be met.

3. Implementation Phase [WASI Subgroup]

Entry requirements:

- The portability criteria must be either met or there must be a plan for how they're expected to be met.
- All dependencies of the wit descriptions must have reached phase 3.

During this phase, the following proceeds in parallel:

- Implementations are built
- Toolchains, libraries, and other tools using the API are built
- Remaining open questions are resolved.
- The plan for satisfying the portability criteria is followed, though the plan may change over time.

Consensus

<https://github.com/WebAssembly/meetings/blob/main/process/consensus.md>

The chair asks all participants to express their opinion to the question, asking in turn whether they are **Strongly For, For, Neutral, Against, or Strongly Against**. Participants vote for a single option by raising their hand, or abstain entirely. Aggregate votes are recorded by the note-taker.

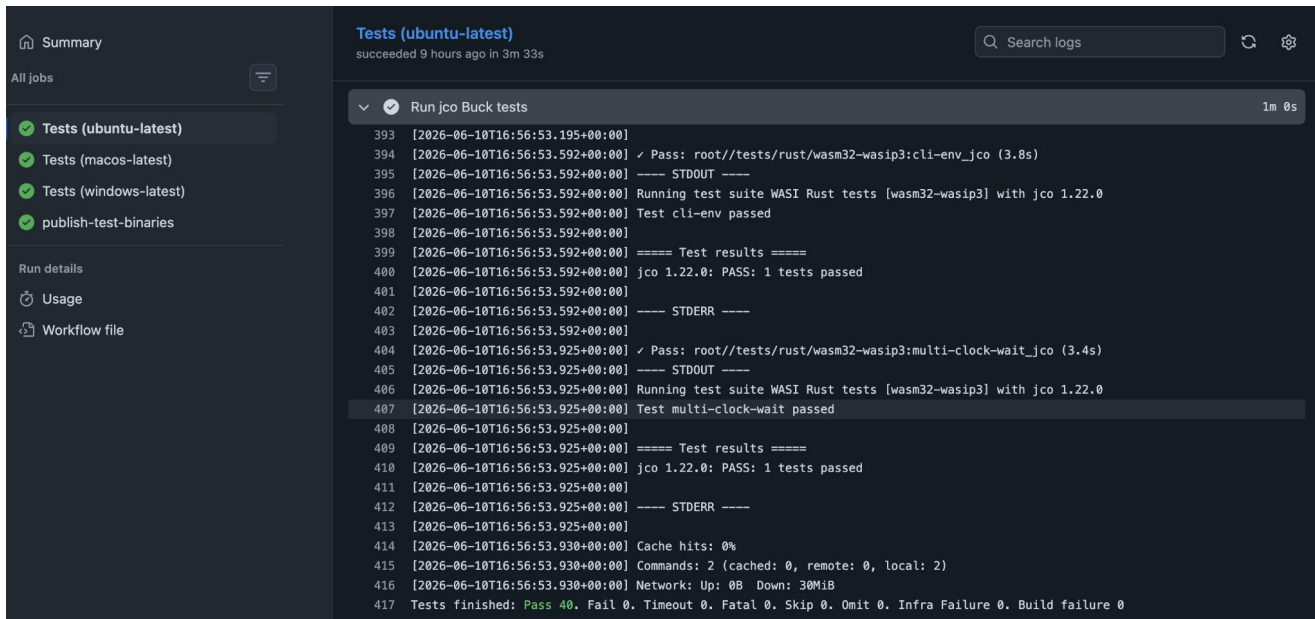
The chair determines whether consensus was reached.

Wasmtime Implementation Status - **Complete**

- **WebAssembly/wasi-testsuite: WASI Rust tests [wasm32-wasip3]** passing
Full CI integration for MacOS, Linux & Windows
- Platforms tested in Wasmtime CI:
 - Linux x86_64
 - Linux aarch64
 - Linux s390x
 - Linux riscv64
 - MacOS x86_64
 - MacOS aarch64
 - Windows MSVC x86_64
 - Windows MinGW x86_64

JCO Implementation Status - Complete

- **WebAssembly/wasi-testsuite: WASI Rust tests [wasm32-wasip3] passing**
Full CI integration for MacOS, Linux & Windows



The screenshot displays a CI/CD pipeline interface with a dark theme. On the left, a sidebar shows a list of jobs: 'Tests (ubuntu-latest)', 'Tests (macos-latest)', 'Tests (windows-latest)', and 'publish-test-binaries'. The 'Tests (ubuntu-latest)' job is selected and expanded, showing its details. The main panel displays the test results for 'Run jco Buck tests', which succeeded 9 hours ago in 3m 33s. The log shows two test suites: 'WASI Rust tests [wasm32-wasip3]' and 'WASI Rust tests [wasm32-wasip3]'. Both suites passed, with 1 test passed each. The log also includes timestamps, status messages, and a summary of the test results.

```
393 [2026-06-10T16:56:53.195+00:00]
394 [2026-06-10T16:56:53.592+00:00] ✓ Pass: root//tests/rust/wasm32-wasip3:cli-env_jco (3.8s)
395 [2026-06-10T16:56:53.592+00:00] ---- STDOUT ----
396 [2026-06-10T16:56:53.592+00:00] Running test suite WASI Rust tests [wasm32-wasip3] with jco 1.22.0
397 [2026-06-10T16:56:53.592+00:00] Test cli-env passed
398 [2026-06-10T16:56:53.592+00:00]
399 [2026-06-10T16:56:53.592+00:00] ===== Test results =====
400 [2026-06-10T16:56:53.592+00:00] jco 1.22.0: PASS: 1 tests passed
401 [2026-06-10T16:56:53.592+00:00]
402 [2026-06-10T16:56:53.592+00:00] ---- STDERR ----
403 [2026-06-10T16:56:53.592+00:00]
404 [2026-06-10T16:56:53.925+00:00] ✓ Pass: root//tests/rust/wasm32-wasip3:multi-clock-wait_jco (3.4s)
405 [2026-06-10T16:56:53.925+00:00] ---- STDOUT ----
406 [2026-06-10T16:56:53.925+00:00] Running test suite WASI Rust tests [wasm32-wasip3] with jco 1.22.0
407 [2026-06-10T16:56:53.925+00:00] Test multi-clock-wait passed
408 [2026-06-10T16:56:53.925+00:00]
409 [2026-06-10T16:56:53.925+00:00] ===== Test results =====
410 [2026-06-10T16:56:53.925+00:00] jco 1.22.0: PASS: 1 tests passed
411 [2026-06-10T16:56:53.925+00:00]
412 [2026-06-10T16:56:53.925+00:00] ---- STDERR ----
413 [2026-06-10T16:56:53.925+00:00]
414 [2026-06-10T16:56:53.930+00:00] Cache hits: 0%
415 [2026-06-10T16:56:53.930+00:00] Commands: 2 (cached: 0, remote: 0, local: 2)
416 [2026-06-10T16:56:53.930+00:00] Network: Up: 0B Down: 30MiB
417 Tests finished: Pass 40. Fail 0. Timeout 0. Fatal 0. Skip 0. Omit 0. Infra Failure 0. Build failure 0
```

wasi-cli is at Phase 3

Portability Criteria

- WASI CLI must have host implementations which can pass the testsuite on at least Windows, macOS, and Linux.
- WASI CLI must have at least two complete independent implementations.

Depends on:

- wasi-clocks
- wasi-random
- wasi-filesystem
- wasi-sockets

Vote to include the wasi-cli world in WASIP3

- Strongly For (SF)
- For (F)
- Neutral (N)
- Against (A)
- Strongly Against (SA)

wasi-http is at Phase 3

Portability Criteria

- WASI-http must have at least two complete independent implementations demonstrating embeddability in a production HTTP server context.

Depends on:

- wasi-clocks
- wasi-random
- wasi-filesystem
- wasi-sockets
- wasi-cli

**Vote to include the
wasi-http/{client,middleware,service}
world in WASIP3**

- Strongly For (SF)
- For (F)
- Neutral (N)
- Against (A)
- Strongly Against (SA)

WASI 0.3.0

Launch criteria

WASI 0.3.0 will be considered launched when at least two independent proposals which define worlds, and all their dependencies, have met the requirements for inclusion, and the WASI Subgroup has voted to launch it.

Vote to launch WASIP3

- Strongly For (SF)
- For (F)
- Neutral (N)
- Against (A)
- Strongly Against (SA)

Appendix

Breaking changes on proposals at phase 3

Our approach in the WASI SG follows the WebAssembly Community Group's handling of the Exception Handling proposal, which took a breaking change at Phase 3

(reintroducing `exnref`; replacing `try/catch` with `try_table/throw_ref`)

via a recorded SG poll in the [October 2023 CG meeting][EH vote], without a phase reset, and preserved the prior design under a ``legacy/`` path in the [exception-handling repository][EH legacy].

[EH vote]: <https://github.com/WebAssembly/meetings/blob/main/main/2023/CG-10.md>

[EH legacy]:
<https://github.com/WebAssembly/exception-handling/tree/main/proposals/exception-handling>